Résultats attendus des tests

Projet composants et services web 2026 · Thème 24 : gestion du cycle de vie d'une API avec WSO2 API Manager

1. Comment les tests sont exécutés

Tous les appels passent par la passerelle WSO2 : `https://localhost:8243/structures/<version>/...`. Les codes d'erreur WSO2 sont ceux de la documentation officielle « Error Handling » de WSO2 API Manager.

- **Automatique** : `python tests/e2e.py` rejoue tous les scénarios, y compris les changements d'état, et produit le rapport des résultats obtenus.
- **Manuel** : collection Postman `tests/postman_collection.json` (dossiers 0 à 5), exécutable aussi avec Newman.
- **Backend seul** : `cd api && pytest -v` (30 tests, sans WSO2).

2. Scénarios et résultats attendus

N°	Scénario	Préparation	Requête	Résultat attendu
Scénarios fonctionnels (chaîne client → WSO2 → FastAPI → PostgreSQL)				
T1	Liste paginée	aucune	`GET /structures/1.0.0/structures?page=1&size=10`	**200**, 10 éléments, `totalPages = ceil(total / size)`, aucun champ de la v2
T2	Détail d'une structure	aucune	`GET .../structures/5`	**200**, `id = 5`
T3	Structure inexistante	aucune	`GET .../structures/9999`	**404**, `STRUCTURE_NOT_FOUND`, format `{code, message, details, path, timestamp}`
T4	Paramètre invalide	aucune	`page=-1`, `size=abc`, `type=HOPITAL`	**400**, `VALIDATION_ERROR`, champ fautif indiqué dans `details`
Sécurité, droits d'accès et quota				
T5	Jeton absent	aucune	sans en-tête `Authorization`	**401**, code WSO2 **900902**
T6	Jeton invalide	aucune	`Authorization: Bearer faux`	**401**, code WSO2 **900901**
T7	Application non abonnée	jeton de AppNonAbonnee	`GET .../structures`	**403**, code WSO2 **900908**

N°	Scénario	Préparation	Requête	Résultat attendu
T7b	Souscription bloquée	`python wso2/wso2ctl.py block-sub`	`GET .../structures`	**401**, code WSO2 **900907** (débloquer ensuite : `unblock-sub`)
T8	Dépassement de quota	plan `Plan10ParMinute`	11 appels ou plus en moins d'une minute	**429**, code WSO2 **900804** (quota de la souscription dépassé ; le rejet peut arriver un peu après le 11e appel, car WSO2 décide de façon asynchrone)
Cycle de vie de l'API				
T9	Retrait temporaire de publication	`python wso2/wso2ctl.py block`	`GET .../structures`	**503**, code WSO2 **700700** ; GET `http://localhost:8000/health` répond toujours 200
T10	Arrêt technique	`docker compose stop api` (API toujours PUBLISHED)	`GET .../structures`	**500**, code WSO2 **101503** (connexion au backend impossible)
T11	Nouvelle version	`python wso2/wso2ctl.py new-version`	`GET /structures/2.0.0/structures/1`	**200** avec `district`, `capacite_lits`, `services` ; la v1 passe en DEPRECATED et répond encore
T12	Republication	`python wso2/wso2ctl.py republish`	`GET .../structures`	**200**, sans autre reconfiguration

3. Retrait de publication et arrêt technique : deux états distincts

T9 et T10 montrent que retirer l'API de la publication et subir une panne du backend sont deux situations différentes, que WSO2 traite différemment.

Critère	T9 - Retrait de publication	T10 - Arrêt technique
Origine	Décision de gestion dans le Publisher	Panne ou arrêt du conteneur backend
État du cycle de vie WSO2	BLOCKED	PUBLISHED (inchangé)
Qui répond au client	La passerelle, sans appeler le backend	La passerelle, après échec de connexion au backend
Réponse	503, code **700700**	500, code **101503**
Backend `/health`	200 (il tourne)	inaccessible
Retour à la normale	`python wso2/wso2ctl.py republish`	`docker compose start api`

4. Tests du backend seul (sans WSO2)

`cd api && pytest -v` : **30 tests** couvrant la pagination, les filtres, la recherche, les erreurs 400, 404 et 503 et la conformité des réponses aux contrats OpenAPI v1 et v2. Ils demandent une base chargée : avec Docker, DATABASE_URL=`postgresql://sante:sante@localhost:5433/sante_db`.